

COSC2769/2808 – Full Stack Development

Software Requirements Specification

Version 1.0

TicTacToang – Online TicTacToe Gaming Platform

Due Date: 18:00 on 6th May 2026

Table of Contents

Introduction.....	2
Contribution Requirements	2
Software Development Methodology Requirements	2
Project Report Requirements.....	4
Frequent Usage of GitHub	4
Architecture Requirements.....	5
Functional Requirements.....	7
1/ Player Registration	7
2/ Login	8
3/ Profile Management	9
4/ TicTacToe Game.....	10
5/ Premium Subscription.....	13
6/ Admin Functionality	14
Deployment Requirements.....	14
Presentation Requirements	Error! Bookmark not defined.
Penalty	15

Introduction

For generations, Tic-Tac-Toe has served as the universal game to strategic thinking, a simple grid that conceals a world of logic, anticipation, and strategy. **TicTacToang** is born from the desire to take this timeless classic and propel it into the modern digital era. It is not merely a game; it is a high-performance web platform designed to bring people together, whether they are sitting side-by-side or separated by oceans. By blending nostalgic gameplay with contemporary web technologies, **TicTacToang** transforms a childhood pastime into a sleek, responsive, and engaging experience.

The mission of **TicTacToang** is to provide a seamless arena for every type of player. From the solo strategist testing their wits against a machine to friends battling for dominance on a shared screen or across the global web, the platform is built for speed, reliability, and joy. Through this project, we aim to redefine how a "simple" game feels in the hands of a user, ensuring that every move is thrilling and every victory feels earned. Welcome to **TicTacToang**, where every "X" and "O" tells a story of connection and strategic mastermind.

You are highly recommended to analyze the requirements of all three difficulties to decide the target complexity, capability, flexibility, and goals of your system architecture and application.

Contribution Requirements

Every team member is expected to contribute to ALL the following tasks by producing recognizable deliverables or providing substantial feedback:

1. Design Data Model and Sequence Diagram
2. Design System Architecture
3. UX/UI Design
4. Technical Report
5. Front-end AND Back-end Development

For development tasks, each member is expected to contribute either to individual or pair programming. The amount of fair contribution is determined by the complexity of the supported features and average workload of the entire team. **Final contribution assessment and weight distribution of grades for group assignments will be made at the discretion of the course coordinator**

If there exists any group member not completing their allocated tasks, inform your Course Coordinator as soon as possible so we can intervene before the problem escalates.

Software Development Methodology Requirements

This section specifies the compulsory methodological framework and teamwork standards required for the execution and delivery of the **TicTacToang** project.

Collaborative Development: All teams are required to adopt an **iterative development** approach, such as SCRUM or Kanban, to enable periodic collaboration and advance the project requirements incrementally.

To ensure consistency, transparency, and traceability, every team must utilize **GitHub** and its **project management tool** to rigorously track all project tasks. This **GitHub** board must document individual tasks, track team commitment, progress, and verify the completion status of all work items.

Each development Team must include the **lecturer** as a participant on their digital project board to enable real-time monitoring and support.

Communication and Review Cycle: A formal, recurring review cycle is mandatory to maintain alignment and quality. **Every one or two weeks**, each team is required to schedule and conduct a combined **Sprint Review and Retrospective Meetings**.

Each team must participate in scheduled meetings with the lecturer in **Week 2, Week 6, and Week 9**. These sessions serve as critical milestones with the live introduction and setup of GitHub project management board in Week 2. In Week 6, every team must **present current progress on system design, UI design, and data models**. In Week 9, every team reports their **progress in system development**. In Weeks 6 and 9, teams discuss lessons learned and experiences gained, as well as proactively identifying and addressing any project impediments or risks with the instructor.

Professionalism: Teamwork performance will be comprehensively evaluated based on three core dimensions: **Contribution to Development** (code quality and feature implementation), **Management Adherence** (quality of board updates and task tracking), and **Involvement** (active participation in all meetings and discussions).

Meeting Attendance Policy: Punctuality and presence at scheduled review meetings are **paramount**. Should any team member be unable to attend a meeting with the lecturer, they are required to provide notification **as soon as possible or at least 24 hours in advance** with a **valid reason** and **supporting evidence**.

Any absence **without prior notification AND valid justification** will automatically result in a **substantial penalty** applied to the individual's teamwork assessment criteria.

Project Report Requirements

In this section, every team is required to document the **Application Design and Implementation** in a maximum **25-page report**. The page count does **NOT include the cover page and table of content**. The Report Template and Required Content are included in the Group Project Assignment on Canvas.

You shall analyze the following technologies to design your architecture for leveraging the potential of the chosen tech stack.

a/ Frontend

- **React-based framework:** e.g., React, Vite
- **Vanilla (Plain) HTML, CSS, and Javascript**
- **Typescript** (optional)

b/ Backend

- **MEN Stack:** MongoDB, ExpressJS, NodeJS

c/ Deployment: Local PC or Cloud (E.g., Render)

Frequent Usage of GitHub

Your team must use GitHub as the only tool to store your deliverables, including diagrams, reports, database, and code base. You are required to use GitHub in a **professional, iterative, and active** approach by frequently pushing your primitive, partially completed, or finalized deliverables into the repositories, and use appropriate communication language.

Additional Note:

- 1) NOT Using GitHub frequently during the project results in up to 5 points penalty to the assessment
- 2) In your report, submit the proof of GitHub contribution throughout the project duration
- 3) In Canvas, submit the zip file of your GitHub repository.

Missing either 2) or 3) **results in three points penalty each.**

Architecture Requirements.

You are strongly recommended to read through requirements of ALL three Levels and the rubric to decide your project goals and plans.

You are required to implement the architecture and features using the following technologies:

a/ Frontend

- React-based framework: e.g., React, Vite
- Vanilla CSS or CSS framework (e.g., Bootstrap, Tailwind)

b/ Backend

- Database: MongoDB
- Server: ExpressJS, NodeJS

c/ **Deployment:** Local Machine or Cloud Deployment (e.g., Render)

Role Specification

- **PLAYER:** Registered Individuals who want to play TicTacToe
- **ADMIN:** Individuals managing accounts and game rooms.

Level	Feature ID	Description
Simplex	A.1.1	Apply N-Tier (Layer-based) Architecture to divide your front-end AND back-end into different layers. Each layer has a designated concern and stores all classes addressing that concern across different business features.
	A.1.2	Design a NodeJS Backend Layer Hierarchy with Route , Controller , Service (Business Logic), Repository , and Model (Data Access) Layers. To reuse query operations <u>in the back-end</u> and separate query execution from Data Access, you are required to implement a <u>Repository</u> Layer , where queries statements are defined and executed.
	A.1.3	A typical Frontend Layer Hierarchy is Page , Component , Hook , Service , and Reusable Component . The latter defines common display elements (e.g., Buttons, Labels) as configurable components in a separate folder, so they can be reused by different pages without redefining the style again.

	A.1.4	Any Feature that does not follow the Tier structure or is used globally shall be organized separately. E.g., Configuration, Filter.
Medium Backend	A.2.1	Apply Modular Monolith Architecture to your back-end. Each module has a bounded context to handle a specific (group of) business feature. A module stores all classes following the N-Tier Layers to provide the business feature.
	A.2.2	Within a Module, the access rules shall go from one layer to another. Specifically, the Presentation Layer shall communicate with the Business Logic Layer which calls Repository method to execute queries. Presentation, Business Logic, and Repository Layers use the Model to read, write, and transfer data.
	A.2.3	The Backend must implement a dedicated middleware layer to authorize users when they access a Backend API to prevent privilege escalation. For example, a Player can NOT access the APIs dedicated for Admin only.
Medium Frontend	A.2.a	The front-end must contain global configurations of Backend API routes , grouped by API business domains, e.g., authentication config, game config, profile config.
	A.2.b	The front-end must contain a REST HTTP Helper Class to define a utility function for sending REST Requests such as GET, POST, PUT, PATCH, and DELETE, along with parameters of request headers and request body. This utility function also returns the data, response status code (e.g., <u>200</u> for OK, <u>404</u> for Not Found), and response headers (if applicable)
	A.2.c	The frontend must authorize user by roles when they access a page, such that Players can NOT access Admin pages , and a Player can NOT access sensitive information of another Player.
Ultimo Backend	A.3.1	Each module exposes external services via interfaces . Other modules must NOT call the Service Layer of a module to invoke its services.
	A.3.2	The backend must apply DTO to present only necessary data when responding to a request from the front-end AND other modules . For example, when showing the PLAYERS matching the search criteria, their budget shall NOT be exposed in the response.

Ultimo Frontend	A.3.a	Componentize your Front-end, such that each component has a package containing the following elements: 1. Component's presentation in HTML or JSX 2. Sub-components, if applicable 3. Hooks (Event Handlers) of the component 4. Back-end service calls 5. CSS styling, if applicable All the five elements are NOT placed in the same Component JavaScript file. These five elements shall be placed in separate files under the same package.
	A.3.b	The Component and Hook are not tightly coupled, so the Component's behaviors are customizable.
	A.4.b	Support Responsive Design for the <u>User Profile</u> and <u>All Admin</u> UIs.

Functional Requirements

1/ Player Registration

Level	Feature ID	Description
Simplex	1.1.1	The system shall provide a registration form requiring <u>Username</u> , <u>Email</u> , <u>Password</u> , <u>Confirm Password</u> , and <u>Country</u> as mandatory fields.
	1.1.2	The system shall enforce a unique email constraint , ensuring NO two users share the same email address in the database.
	1.1.3	The password must be hashed when storing in the backend. Note: hashing is different than encryption. If you encrypt the password, no point will be granted for this requirement.
	1.1.4	The Country input field must be a selectable option (e.g., a dropdown list) instead of a direct text input field.
Medium	1.2.1	The system shall validate password strength against the following criteria: a) At least 8 characters. b) At least 1 number. c) At least 1 special character (e.g., \$#@!). d) At least 1 capitalized letter.

	1.2.2	The system shall validate the email syntax to ensure it meets standard formatting requirements, including a) Contains exactly one '@' symbol. b) Contains at least one '.' (dot) after the '@' symbol. c) The total length of the email address must be less than 255 characters. d) No spaces or prohibited characters (e.g., () ;).
	1.2.3	The system shall validate the Username syntax that allows only English alphabets, numbers, underscore (_) and hyphen (-).
Ultimo	1.3.1	All input validations specified above must be performed on both the frontend for quick user feedback and the backend for security. Any validation error on the frontend must be clearly displayed to the end-user. Any validation error on the backend must display the error message in the response . Each error message in the frontend or backend describes the error, cause of error, and examples of valid inputs .

2/ Login

Level	Feature ID	Description
Simplex	2.1.1	The system shall authenticate the player and admin using the username or email, and password. The system must apply the hashed password in authenticating the user credentials.
Medium	2.2.1	The system shall implement a mechanism to prevent brute-force attacks on the login endpoint, blocking the authentication for that account after 5 failed login attempts within 60 seconds.
	2.2.2	Upon successful login, the backend shall provide a mechanism to inform the frontend of the user identity information (e.g., user ID, Role. This user identity is verified on the backend to authorize the user's requests of accessing backend APIs.
	2.2.3	When the user logs out, the system shall invalidate/erase the user identity in the storage media used to implement requirement 2.2.2.
Ultimo	2.3.1	Instead of implementing 2.2.2, upon successful login, the backend shall generate a JSON Web Signature (JWS) token. This token must contain user identity information (e.g., user ID, Role) to authorize the user's requests for accessing backend APIs.

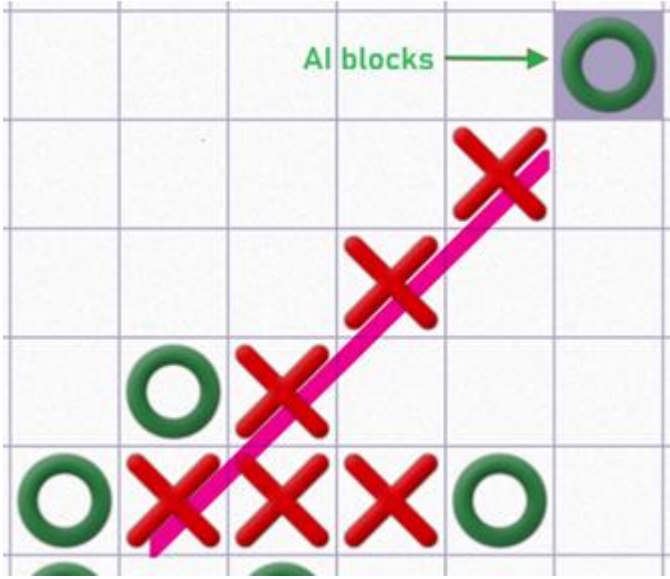
	2.3.2	Instead of implementing 2.2.3, the system shall invalidate and revoke the user's JWS token when the user explicitly logs out or after the defined expiration time.
--	--------------	--

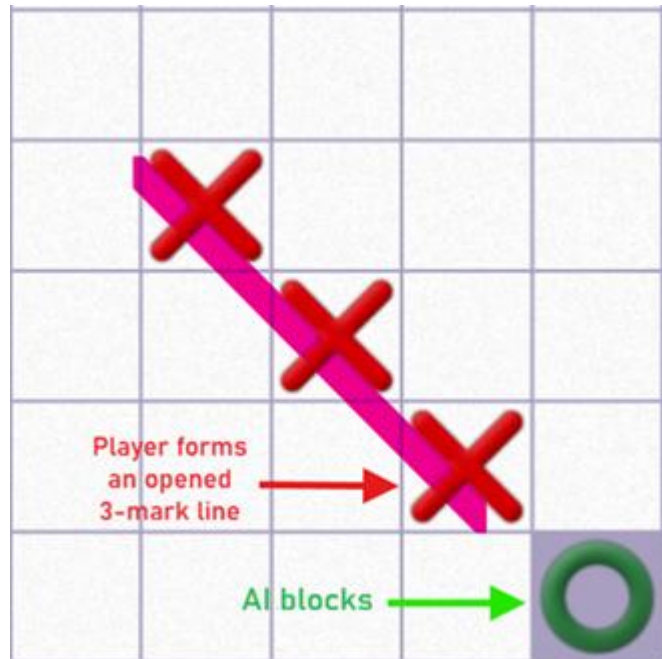
3/ Profile Management

Level	Feature ID	Description
Simplex	3.1.1	Players shall be able to edit their core contact information : Email, Username, Password, and Country.
	3.1.2	The Player can see the list of game sessions that they played in the past. Each session shows the start time, end time, game type (Single Player, Two Players, Online Match), result, and players in the game. For single player games, include the AI bot name as the other player.
Medium	3.2.1	The Player shall be able to upload a Logo picture . The system must automatically resize the image to a defined standard size for optimal display performance.
	3.2.2	The system shall enable searching for game session(s) from the game session number or Player 2 Name. The search is case-insensitive and is a pattern search . E.g., if searching for Player 2 name by “na”, then any game whose Player 2 name contains “na” shall be listed, e.g., Anna, Nam, BeNam173. The search can be done on either Frontend or Backend .
Ultimo	3.3.1	The system shall enable filtering of a game session by date range, result (win, lose, aborted), game type (Single Player, Two Player, or Online Match) and sort by date in ascending or descending order. The sorting and filtering can be done in either backend or frontend.

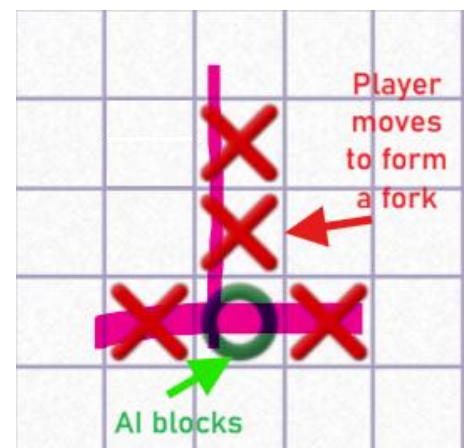
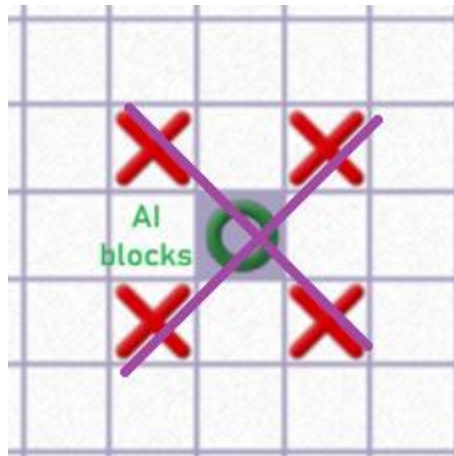
4/ TicTacToe Game

Level	Feature ID	Description
Simplex	4.1.1	The system shall allow a two-player game on a single PC . Only a registered user can initiate the game, and the system must prompt for the second player's name .
	4.1.2	The default board size shall be 10x10 (100 square grids). The board must feature a single background color and clear borders for each cell.
	4.1.3	Players must use distinct markers (e.g., 'X' for Player 1 and 'O' for Player 2). A cell only contains one mark from a player throughout the game.
	4.1.4	A player wins by placing five adjacent marks horizontally, vertically, or diagonally. The system must record the match start time, end time, participants, and results in the backend . The frontend must indicate the 5-mark line of the victor and state who wins the game
	4.1.5	The system shall allow any player to abort the game at any time. In this case, no winner or loser shall be recorded.
	4.1.6	The system shall allow the initiator to choose who goes first .
Medium	4.2.1	The frontend shall allow customization of game properties, including: - Selecting from 3 different board styles - Toggling between 10x10 and 15x15 board sizes - Choosing from 6 distinct markers. Note: Avoid taboos and offensive icons (e.g., cows or bulls are holy in India, so you should avoid marks containing them). The system must display the avatar and selected mark of each player during the game.
	4.2.2	The system shall provide a Single Player mode against an AI with three difficulty levels: Easy (4.2.3), Medium (4.2.4), and Hard (4.2.5) .

	When recording the match, give a distinct name for different AI levels. The names shall clearly express the AI difficulty level. E.g., Jeremy (Easy); Bot (Medium).
4.2.3	The Easy AI shall randomly choose an empty cell immediately adjacent to the player's last move.
4.2.4	The Medium AI must block the player from completing the following patterns: 1) Any 5-mark line  2) Any 4-mark line opened on both ends



- 3) **Any fork formation** (two crossing 3-mark lines) having each line opened on both ends



4.2.5

The **Hard** AI must perform all defensive actions from Requirement 4.2.4 and be able to attack by **completing its own 5-mark line** whenever the opportunity exists.

4.2.6

The system applies an **animation** with motion and color effect to announce the winner on the screen.

Ultimo	4.3.1	The system shall support real-time network play between two players on different PCs. This requires synchronized game states over the network via web socket. The system shall have an arena to display all online game rooms. A new game room created by a player is added to this arena. When the second player joins the game room, the system must inform the first player of the new player. The game starts when the second player chooses their mark.
	4.3.2	The system shall support real-time chat between two players on different PCs while they are playing an Online Multiplayer game
	4.3.3	This feature is available only for Premium User (See Requirement 6) The system must record every move in a match. Players shall be able to open a match record and use a replay interface with Pause, Resume, Forward, and Backward controls to review each step. To enable this feature, the system shall use algebraic notation to display the coordinate axes on the board, regardless of the game mode. One way is using the <u>King Chess convention</u>, i.e., horizontal cell positions are named after alphabets, while vertical cell positions are named after numbers. For example, if Player 1 chooses c2, it is in the third column from the left (c) and second row from the bottom (2). Players can watch a match record of any past game sessions shown in their Profile (See requirement 3.2.2)

5/ Premium Subscription

Level	Feature ID	Description
Simplex	5.1.1	The system shall simulate a monthly subscription feature where Player can pay a fee of 10 USD using a local wallet system. The Player first deposits money into their wallet. Then they will use the money to pay for the monthly subscription.

	5.1.2	The system must notify the player via email when the payment is successfully processed.
Medium	5.2.1	The system shall provide a monthly subscription feature where company can pay a fee of 10 USD using a third-party payment system (e.g., Stripe, Paypal). The system must record and display the premium status on the Player Profile page.

6/ Admin Functionality

Level	Feature ID	Description
Simplex	6.1.1	The system shall provide an Admin interface to see all Players in the system. Each player record shows the Username, Email, Premium Status, and Account Status .
Medium	6.2.1	The system shall enable Admin to deactivate and reactivate a Player Account . Upon deactivation, the user can NOT login into the system until being activated again.
Ultimo	6.3.1	The admin shall provide an Admin interface to list all <u>online</u> game rooms . Each room shows the room number, Player 1 name, Player 2 Name, start time, and end time .
	6.3.2	For convenience, the system should provide a search for game sessions by <u>room number</u> or <u>player name</u> .
	6.3.3	The system shall allow an admin to close any active game room .

Deployment Requirements

Level	Feature ID	Description
Simplex	D.1.1	The frontend and backend systems are deployed in development environment.
Medium	D.2.1	The system is deployed on the cloud (e.g., Render) and accessible globally

Project Interview

After the project submission. The teaching team will schedule meetings to interview each team. Each member will be questioned about their contribution to the development, understanding of architecture, and rationale of applying a programming concept.

Members who cannot explain or respond to the lecturer's questions will be penalized by deducting the corresponding grade on the frontend, backend, or architecture.

To ensure a high-quality presentation and to prove the full functionality of the **TicTacToang** platform, each team must prepare a "Gold Data Set" prior to the demo. **The data must be pre-loaded into the database** so that the team can focus on feature demonstration rather than manual data entry.

Required User Accounts

Teams must have the following accounts ready for immediate login:

- **Admin Account:** A pre-defined account with administrative privileges to demonstrate the Administration Portal (Feature 6).
- **Player A (Premium):** A registered player with an active subscription to demonstrate real-time notifications, custom marks, and replay features.
- **Player B (Standard):** A registered player used for local and online multiplayer testing.

Profile & Asset Preparation

To showcase the **Medium** tier customization features, the following assets must be pre-configured:

- **Avatars:** Professional or themed avatar images (PNG/JPG) uploaded for Player A and Player B.
- **New Player Avatar:** Use a different avatar image for your newly created Player

Penalty

P1) NOT Using GitHub frequently during the project results in up to 5 points penalty of your assessment grade.

P2) In your Contribution Declaration Sheet, submit the proof of GitHub contribution. You will need to **set your repository to public** to collect the contribution data. State the name of every member if the GitHub username does not contain your first name AND last name. **Only publish your repository at most 24 hours before submission**. **Missing the GitHub contribution results in 3 points penalty**.

P3) NOT submitting the **zip file** of your team final deliverable on GitHub results in **3 points penalty**

End of Requirement Specifications